

MAX6577 is susceptible to noise generated both internally and externally. The effects of external noise can be minimised by connecting a 0.01µF ceramic bypass capacitor close to power-supply pin 1 of IC2. The output signal from MAX6577 to the microcontroller should be connected through a short length of a shielded wire. The sensor output (pin 6) is interfaced to the microcontroller using port pin PD6.

A 16-character×2-line LCD module is interfaced to the microcontroller in 8-bit mode and the data bits are interfaced through port C (PC0 through PC7). Pins PA6 and PA7 of port A are used as enable (EN) and register-select (RS) pins of the LCD module. The LCD's pin R/W is pulled 'low' by connecting it to ground permanently and hence the LCD unit is always in 'write' mode. The contrast of the LCD can be controlled by preset VR1.

A push-to-on switch (S2) is interfaced with PA3 of the microcontroller. When it is momentarily pressed, the minimum and maximum temperature measured/recorded by the circuit during its operation are displayed. When pressed again, it displays only the currently monitored temperature; but in the background both minimum and maximum temperatures are also recorded. The temperature is displayed in Centigrades; the software can easily be modified to display the temperature in Fahrenheit. Switch S1 is used for manual reset.

AVR ATmega16 microcontroller has been used as the control and processing unit. For the given circuit and software, it uses a 1MHz internally generated clock. To activate the internal oscillator, the fuse bytes of the microcontroller have to be programmed as follows:

Fuse low byte = D1

Fuse high byte = 99

By default, the JTAG interface of the microcontroller is enabled, and has to be disabled for proper functioning, especially of the LCD module.

Fig. 3 shows the power supply circuit. The 230V AC mains is stepped

TABLE II Functions of Source Program Files	
File name	Function
pwm.c	Contains the code to initialise the various ports and calculate the pulse width of the temperature sensor's signal and subsequently temperature
Lcd.c	Contains the code to control the functioning of the LCD
Key.c	Contains the code to get the status of the push-to-'on' switch
Lcd.h	Contains the code that describes the various interface signal names used and their respective declarations; contains the global declarations and sub-routines which are used by other files

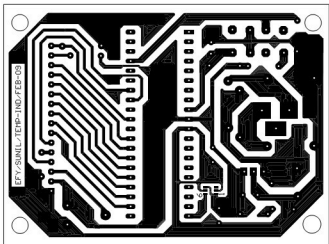


Fig. 4: Actual-size, single-side PCB for the microcontroller-based temperature indicator using PWM temperature sensor

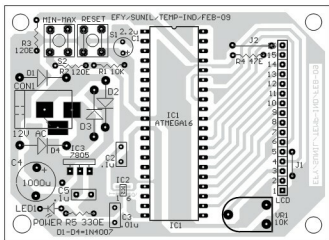


Fig. 5: Component layout for the PCB

down by transformer X1 to deliver the secondary output of 9V, 500 mA. The transformer output is rectified by a full-wave bridge rectifier comprising diodes D1 through D4, filtered by capacitor C4 and then regulated by IC 7805 (IC3). Capacitor C5 bypasses the ripples present in the regulated power

detected, the timer is enabled, and at the first falling edge of the sensor's signal, the timer is disabled. Now contents of the timer are read, and used for computing the pulse width of the temperature sensor's signal. The timer is re-initialised to the count of 0x000 once its contents are read.

supply. LED1 acts as a power-'on' indicator and resistor R5 limits the current through LED1.

An actual-size, single-side PCB for the microcontroller-based temperature indicator is shown in Fig. 4 and its component layout in Fig. 5.

Temperature measurement is achieved by using internal timer 1 of ATmega16. The timer's counts are used for determining the pulse width of the input signals (and subsequently the signal frequency). The get_Sensor_Data() subroutine repeatedly checks the logic state of the sensor's signal. As soon as the first rising edge of the sensor's signal is